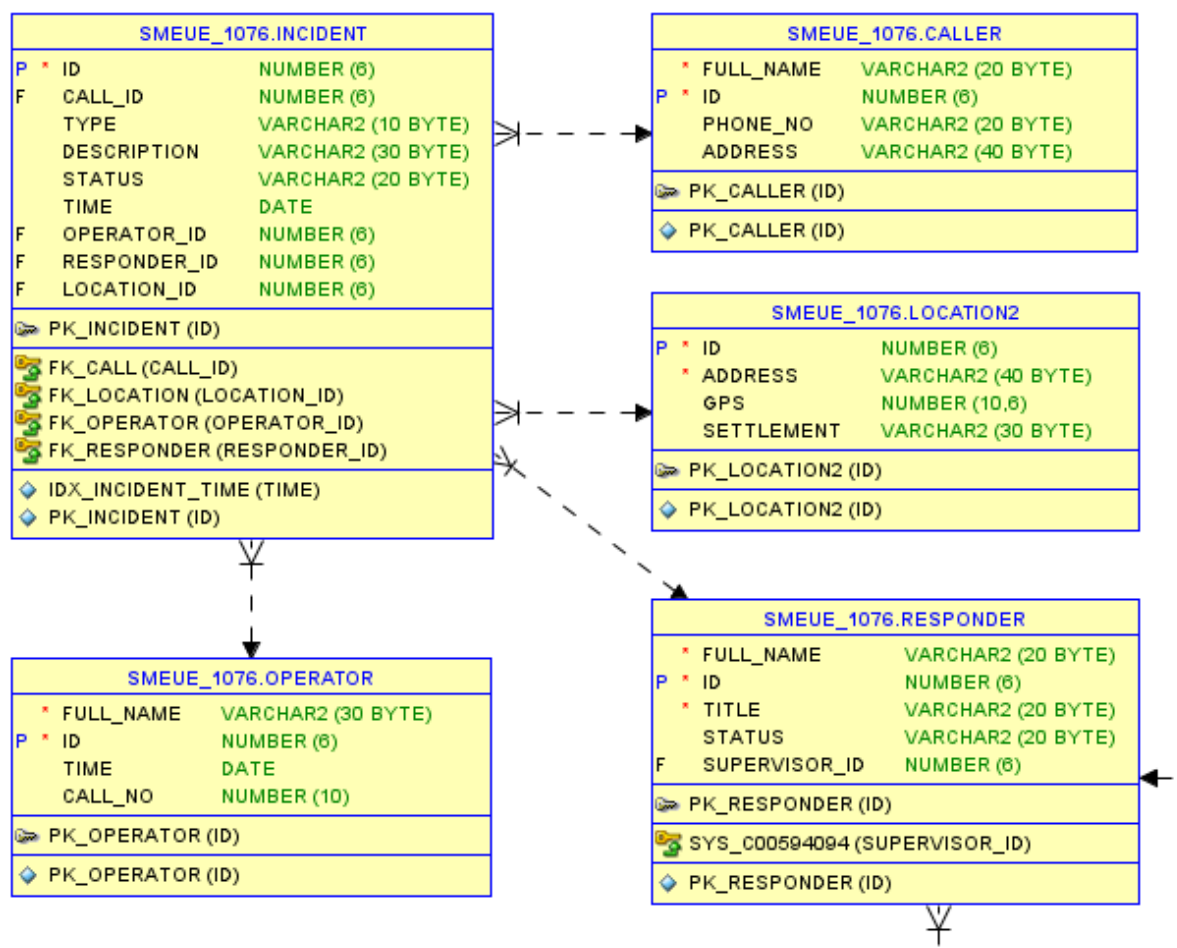


DBMS project

1. Description:

Main topic: Managing emergency calls and responses for Romania's 112 system. The database track emergency calls, callers, responders, incidents and locations.

2. Schema:



3. Possible problems and solutions

What if we lose data? We must have a backup, right?

In general it can happen to anyone to lose data, but in a 112 emergency system there are life-and-death situations where can't be made excuses. Responders involved in missions should be able to access the data at any time.

It is necessary to have a backup so we do the following:

```

BEGIN
    execute immediate 'CREATE table INCIDENT_BACKUP as SELECT * from INCIDENT
    where 1=0';
    execute immediate 'INSERT into INCIDENT_BACKUP SELECT * from INCIDENT';
    DBMS_OUTPUT.PUT_LINE('Backup completed.');
```

END; /

Let's check the pending incident cases...

Each incident is important and we must solve each situation as quickly as possible, but we still have a few cases with pending status, unsolved ones. We must find how many incidents are in such a situation.

```
DECLARE
    v_total NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_total FROM INCIDENT WHERE STATUS = 'Pending';
    DBMS_OUTPUT.PUT_LINE('Total pending incidents: ' || v_total);
END;
/
```

Caller list

If it's needed to check easier the caller names, a caller list can be generated as follows:

```
DECLARE
    TYPE CallerList IS TABLE OF CALLER.FULL_NAME%TYPE INDEX BY PLS_INTEGER;
    callers CallerList;
    i PLS_INTEGER := 0;
BEGIN
    FOR rec IN (SELECT FULL_NAME FROM CALLER) LOOP
        i := i + 1;
        callers(i) := rec.FULL_NAME;
    END LOOP;

    FOR j IN 1..i LOOP
        DBMS_OUTPUT.PUT_LINE('Caller: ' || callers(j));
    END LOOP;
END;
/
```

Name check:

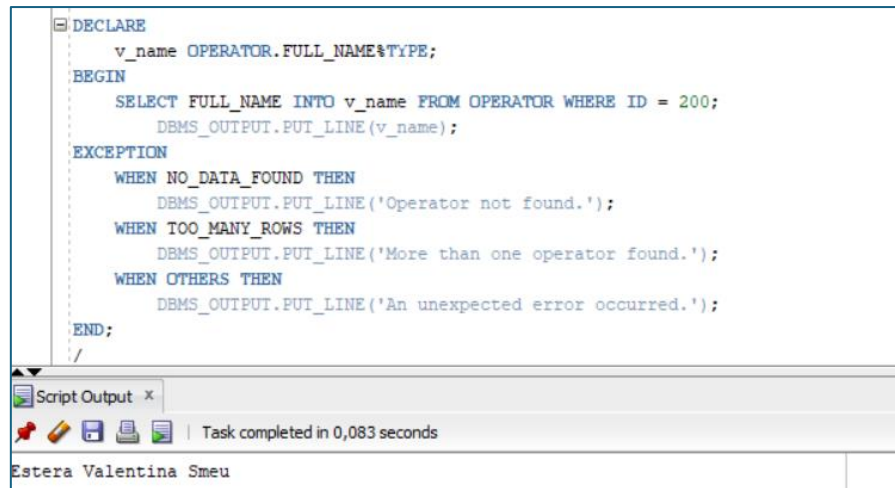
We need to find the name of an operator, but we have a single information, the operator ID, so we do the following:

```
DECLARE
    v_name OPERATOR.FULL_NAME%TYPE;
BEGIN
    SELECT FULL_NAME INTO v_name FROM OPERATOR WHERE ID = 200;
    DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Operator not found.');
```

```
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('More than one operator found.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred.');
```

```
END;
/
```



```

DECLARE
    v_name OPERATOR.FULL_NAME%TYPE;
BEGIN
    SELECT FULL_NAME INTO v_name FROM OPERATOR WHERE ID = 200;
    DBMS_OUTPUT.PUT_LINE(v_name);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Operator not found.');
```

Script Output x

Task completed in 0,083 seconds

Estera Valentina Smeu

Status errors?

We usually check if the status is 'Pending', 'Active' or 'Closed', but what if someone wrote it differently? We have to find out if there is an issue, and if so, where.

```

DECLARE
    e_invalid_status EXCEPTION;
    CURSOR invalid_status_c IS
        SELECT ID, STATUS FROM INCIDENT
        WHERE STATUS NOT IN ('Pending', 'Active', 'Closed');

    v_id INCIDENT.ID%TYPE;
    v_status INCIDENT.STATUS%TYPE;
BEGIN
    OPEN invalid_status_c;
    LOOP
        FETCH invalid_status_c INTO v_id, v_status;
        EXIT WHEN invalid_status_c%NOTFOUND;

        -- Raise custom exception for each invalid status
        RAISE e_invalid_status;
    END LOOP;
    CLOSE invalid_status_c;

EXCEPTION
    WHEN e_invalid_status THEN
        DBMS_OUTPUT.PUT_LINE('Invalid incident status found for ID: ' || v_id
        || ' (Status: ' || v_status || ')');
```

What if we cannot find a person or their name?

We want to find if a caller with a certain ID exists, and if so, it's name, but what if we are not able to?

```

DECLARE
    e_caller_not_found EXCEPTION;
    v_id NUMBER := 800;
    v_name VARCHAR2(20);
BEGIN
```

```

        SELECT FULL_NAME INTO v_name FROM CALLER WHERE ID = v_id;
        IF v_name IS NULL THEN
            RAISE e_caller_not_found;
        END IF;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            DBMS_OUTPUT.PUT_LINE('Caller not found with ID ' || v_id);
        WHEN e_caller_not_found THEN
            DBMS_OUTPUT.PUT_LINE('Custom Exception: No name found for caller
ID. ');
    END;
/

```

Between 'status' and 'ID & type'

We want to find the ID and the type of incidents with the status 'Pending'.

```

DECLARE
    CURSOR incident_cursor(p_status VARCHAR2) IS
        SELECT ID, TYPE FROM INCIDENT WHERE STATUS = p_status;
    v_id NUMBER;
    v_type VARCHAR2(10);
BEGIN
    OPEN incident_cursor('Pending');
    LOOP
        FETCH incident_cursor INTO v_id, v_type;
        EXIT WHEN incident_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE('ID: ' || v_id || ' Type: ' || v_type);
    END LOOP;
    CLOSE incident_cursor;
END;
/

```

Function to retrieve an operator's name

We want to find an operator name based on the ID, and a function which is able to bring it quick is the best option, so we proceed as follows:

```

CREATE OR REPLACE FUNCTION get_operator_name(p_id NUMBER)
RETURN VARCHAR2 IS
    v_name VARCHAR2(30);
BEGIN
    SELECT FULL_NAME INTO v_name FROM OPERATOR WHERE ID = p_id;
    RETURN v_name;
END;
/

```

Incidents count function:

```

CREATE OR REPLACE FUNCTION count_resolved_incidents RETURN NUMBER IS
    v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM INCIDENT WHERE STATUS = 'Resolved';
    RETURN v_count;
END;
/

```

Does a certain caller exists? Check with a function:

```
CREATE OR REPLACE FUNCTION caller_exists(p_id NUMBER) RETURN BOOLEAN IS
    v_dummy NUMBER;
BEGIN
    SELECT 1 INTO v_dummy FROM CALLER WHERE ID = p_id;
    RETURN TRUE;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/
```

Procedure to update an incident status with a known ID

It was made a mistake in the database and a suitable solution must be found for it.

```
CREATE OR REPLACE PROCEDURE active_incident(p_id NUMBER) IS
BEGIN
    UPDATE INCIDENT SET STATUS = 'Active' WHERE ID = p_id;
    COMMIT;
END;
/
```

Assign data

The incident responder is very important, so we want to assign it properly, so we do as follows:

```
CREATE OR REPLACE PROCEDURE assign_responder(p_incident_id NUMBER,
p_responder_id NUMBER) IS
BEGIN
    UPDATE INCIDENT
    SET RESPONDER_ID = p_responder_id
    WHERE ID = p_incident_id;
    COMMIT;
END;
/
```

New incident? Let's update the table!

We have a new incident case and we can insert it's values with a procedure as follows:

```
CREATE OR REPLACE PROCEDURE log_new_incident(p_call_id NUMBER, p_type
VARCHAR2, p_desc VARCHAR2, p_operator_id NUMBER, p_location_id NUMBER) IS
BEGIN
    INSERT INTO INCIDENT(ID, CALL_ID, TYPE, DESCRIPTION, STATUS, TIME,
OPERATOR_ID, LOCATION_ID)
    VALUES (INCIDENT_SEQ.NEXTVAL, p_call_id, p_type, p_desc, 'Resolved',
SYSDATE, p_operator_id, p_location_id);
    COMMIT;
END;
/
```

Let's make a package with functions and procedures

It may happen to need to update an incident's status by getting a caller's full name by ID, but let's display the number of the incidents too!

1 function: gets a caller's full name by ID

2 procedures: updates an incident's status & displays how many total incidents exist

```
CREATE OR REPLACE PACKAGE emergency_pack IS
    FUNCTION get_caller_name(p_id NUMBER) RETURN VARCHAR2;
    PROCEDURE update_incident_status(p_incident_id NUMBER, p_new_status
VARCHAR2);
    PROCEDURE count_total_incidents;
END emergency_pack;
/

CREATE OR REPLACE PACKAGE BODY emergency_pack IS

FUNCTION get_caller_name(p_id NUMBER) RETURN VARCHAR2 IS v_name VARCHAR2(20);
BEGIN
    SELECT FULL_NAME INTO v_name
    FROM CALLER
    WHERE ID = p_id;
    RETURN v_name;

    EXCEPTION
    WHEN NO_DATA_FOUND THEN RETURN 'Caller not found';
END;

PROCEDURE update_incident_status(p_incident_id NUMBER, p_new_status VARCHAR2)
IS
BEGIN
    UPDATE INCIDENT
    SET STATUS = p_new_status
    WHERE ID = p_incident_id;

    DBMS_OUTPUT.PUT_LINE('Incident ID ' || p_incident_id || ' updated to
status: ' || p_new_status);

    EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error updating incident: ' || SQLERRM);
END;

PROCEDURE count_total_incidents IS v_count NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_count FROM INCIDENT;
    DBMS_OUTPUT.PUT_LINE('Total number of incidents: ' || v_count);
END;

END emergency_pack;
/
```

System date of the new inserted value

Who thought a statement-level trigger would be so easy to use in such a case?

```
CREATE OR REPLACE TRIGGER trg_after_incident_insert
AFTER INSERT ON INCIDENT
```

```
BEGIN
    DBMS_OUTPUT.PUT_LINE('A new incident was inserted at ' || SYSDATE);
END;
/
```

Responder's update statement-level trigger

```
CREATE OR REPLACE TRIGGER trg_before_responder_update
BEFORE UPDATE ON RESPONDER
BEGIN
    DBMS_OUTPUT.PUT_LINE('Responder update triggered');
END;
/
```

From 'active' to 'solved'

If in the incident table exists NULL status, it has to be changed with 'Resolved', so we proceed as follows:

```
CREATE OR REPLACE TRIGGER trg_default_incident_status
BEFORE INSERT ON INCIDENT
FOR EACH ROW
BEGIN
    IF :NEW.STATUS IS NULL THEN
        :NEW.STATUS := 'Resolved';
    END IF;
END;
/
```

Medic... Paramedic... well isn't it the same thing?

The title we insert must be eligible. We can make lists with data with the same title, so the way a title is written trully matters.

```
CREATE OR REPLACE TRIGGER trg_validate_responder_title
BEFORE INSERT OR UPDATE ON RESPONDER
FOR EACH ROW
BEGIN
    IF :NEW.TITLE NOT IN ('Paramedic', 'Firefighter', 'Police') THEN
        RAISE_APPLICATION_ERROR(-20001, 'Invalid responder title.');
```

```
    END IF;
END;
/
```

3.2 Third year exercises

What if a responder is overloaded with too many active incidents?

If a responder has more than 3 active incidents assigned at the same time, we risk poor emergency response quality. We need a procedure that checks this condition and, if overloaded, automatically reassign the extra incidents to the responder with the fewest current active engagements. This involves aggregation, cursor iteration, and conditional UPDATE logic.

```
CREATE OR REPLACE PROCEDURE rebalance_responder_load IS
CURSOR overloaded_c IS
SELECT RESPONDER_ID, COUNT(*) AS active_count
FROM INCIDENT
WHERE STATUS = 'Active'
GROUP BY RESPONDER_ID
HAVING COUNT(*) > 3;

v_min_responder_id  RESPONDER.ID%TYPE;
v_min_count         NUMBER;
v_incident_id       INCIDENT.ID%TYPE;

CURSOR extra_incidents_c(p_resp_id NUMBER) IS
SELECT ID FROM INCIDENT
WHERE STATUS = 'Active' AND RESPONDER_ID = p_resp_id
ORDER BY TIME DESC
OFFSET 3 ROWS FETCH NEXT 100 ROWS ONLY;
BEGIN
FOR rec IN overloaded_c LOOP
    -- Find the least loaded responder
    SELECT RESPONDER_ID INTO v_min_responder_id
    FROM (SELECT RESPONDER_ID, COUNT(*) AS cnt
          FROM INCIDENT
          WHERE STATUS = 'Active'
          GROUP BY RESPONDER_ID
          ORDER BY cnt ASC) WHERE ROWNUM = 1;
    -- Reassign extra incidents
    FOR inc IN extra_incidents_c(rec.RESPONDER_ID) LOOP
        UPDATE INCIDENT
        SET RESPONDER_ID = v_min_responder_id
        WHERE ID = inc.ID;
        DBMS_OUTPUT.PUT_LINE('Incident ' || inc.ID || ' reassigned to
responder ' || v_min_responder_id);
    END LOOP;
END LOOP;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Rebalancing complete.');
```

```
END rebalance_responder_load;
/

EXEC rebalance_responder_load;
```


What if we need a full incident report with caller and location info in one view?

If operators need a comprehensive summary of every incident (including the caller's name, address, location settlement, and the responding officer), looking at separate tables is inefficient. We should create a stored function that returns a formatted multi-line text report for a given incident ID, joining across INCIDENT, CALLER, LOCATION2, and RESPONDER. This covers complex multi-table SELECT with exception handling and string formatting.

```
CREATE OR REPLACE FUNCTION get_incident_report(p_id NUMBER)
RETURN VARCHAR2 IS
    v_report    VARCHAR2(4000) := '';
    v_type      INCIDENT.TYPE%TYPE;
    v_status    INCIDENT.STATUS%TYPE;
    v_time      INCIDENT.TIME%TYPE;
    v_caller    CALLER.FULL_NAME%TYPE;
    v_phone     CALLER.PHONE_NO%TYPE;
    v_address   LOCATION2.ADDRESS%TYPE;
    v_settle    LOCATION2.SETTLEMENT%TYPE;
    v_responder RESPONDER.FULL_NAME%TYPE;
BEGIN
    SELECT i.TYPE, i.STATUS, i.TIME,
           c.FULL_NAME, c.PHONE_NO,
           l.ADDRESS, l.SETTLEMENT,
           r.FULL_NAME
    INTO v_type, v_status, v_time,
         v_caller, v_phone,
         v_address, v_settle,
         v_responder
    FROM INCIDENT i
    JOIN CALLER   c ON i.CALL_ID      = c.ID
    JOIN LOCATION2 l ON i.LOCATION_ID = l.ID
    JOIN RESPONDER r ON i.RESPONDER_ID = r.ID
    WHERE i.ID = p_id;

    v_report := '--- INCIDENT REPORT (ID: ' || p_id || ') ---' || CHR(10) ||
               'Type      : ' || v_type      || CHR(10) ||
               'Status    : ' || v_status    || CHR(10) ||
               'Time      : ' || TO_CHAR(v_time, 'DD-MON-YYYY HH24:MI') || CHR(10) ||
               'Caller    : ' || v_caller    || ' (' || v_phone || ')' || CHR(10) ||
               'Location: ' || v_address || ', ' || v_settle || CHR(10) ||
               'Responder: ' || v_responder;

    RETURN v_report;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'ERROR: Incident ID ' || p_id || ' not found or data incomplete.';
END get_incident_report;
/

BEGIN
    DBMS_OUTPUT.PUT_LINE(get_incident_report(1));
END;
/

BEGIN
```

```

    DBMS_OUTPUT.PUT_LINE(get_incident_report(510));
END;
/

```

```

Function GET_INCIDENT_REPORT compiled

ERROR: Incident ID 1 not found or data incomplete.

PL/SQL procedure successfully completed.

--- INCIDENT REPORT (ID: 510) ---
Type   : Medical
Status : Pending
Time    : 26-FEB-2026 11:34
Caller  : Mihai Eminescu (779173280)
Location: Strada Libertatii, Iasi, Iasi
Responder: Stefan Rosu

PL/SQL procedure successfully completed.

```

What if we want to automatically escalate long-pending incidents?

If an incident has had the status 'Pending' for more than 1 day without being picked up, there is a risk that it will be forgotten. We need a scheduled-style procedure that scans all pending incidents, checks their TIME field, and escalates them to 'Active' while logging a message to a dedicated audit trail. This demonstrates DATE arithmetic, bulk processing with a cursor FOR loop, and conditional DML.

```

-- First let's create an audit log table:
BEGIN
EXECUTE IMMEDIATE
'CREATE TABLE INCIDENT_AUDIT (
    AUDIT_ID    NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    INCIDENT_ID NUMBER,
    OLD_STATUS  VARCHAR2(20),
    NEW_STATUS  VARCHAR2(20),
    CHANGED_AT  DATE DEFAULT SYSDATE)';
DBMS_OUTPUT.PUT_LINE('Audit table created.');
```

```

EXCEPTION
WHEN OTHERS THEN
IF SQLCODE = -955 THEN
DBMS_OUTPUT.PUT_LINE('Audit table already exists.');
```

```

ELSE RAISE;
END IF;
END;
/

CREATE OR REPLACE PROCEDURE escalate_stale_incidents(
    p_hours_threshold NUMBER DEFAULT 24) IS v_count NUMBER := 0;
BEGIN
FOR rec IN (
    SELECT ID, STATUS, TIME FROM INCIDENT
    WHERE STATUS = 'Pending'
    AND (SYSDATE - TIME) * 24 > p_hours_threshold)
LOOP
    UPDATE INCIDENT
    SET STATUS = 'Active'

```

```

WHERE ID = rec.ID;

INSERT INTO INCIDENT_AUDIT(INCIDENT_ID, OLD_STATUS, NEW_STATUS)
VALUES (rec.ID, 'Pending', 'Active');

v_count := v_count + 1;
DBMS_OUTPUT.PUT_LINE('Escalated incident ID: ' || rec.ID ||
' (pending since ' ||
TO_CHAR(rec.TIME, 'DD-MON-YYYY HH24:MI') || ')');
END LOOP;
COMMIT;
DBMS_OUTPUT.PUT_LINE('Total escalated: ' || v_count);
END escalate_stale_incidents;
/

BEGIN escalate_stale_incidents;
END;
/

```

What if we need to prevent an incident from being deleted while still Active?

If someone attempts to DELETE a row from the INCIDENT table while that incident still has the status 'Active', this could cause a critical data loss. Active missions would lose their record entirely. We need a row-level BEFORE DELETE trigger that checks the status and raises an application error to block the operation, ensuring data integrity is enforced at the database level rather than relying on the application.

```

CREATE OR REPLACE TRIGGER trg_prevent_active_delete
BEFORE DELETE ON INCIDENT
FOR EACH ROW
BEGIN
IF :OLD.STATUS = 'Active' THEN RAISE_APPLICATION_ERROR( -20002,
'BLOCKED: Cannot delete active incident ID ' ||
:OLD.ID || '. Close or resolve it first.' );
END IF;
END trg_prevent_active_delete;
/

BEGIN
DELETE FROM INCIDENT WHERE ID = 1; -- assuming ID 1 is Active
DBMS_OUTPUT.PUT_LINE('Delete succeeded (incident was not Active).');
EXCEPTION
WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('Caught: ' || SQLERRM);
END;/

```

Let's do a statistical summary! - Incidents grouped by type and status.

For a good management purpose we must understand the distribution of incidents - How many are Pending, Active, or Closed for each type. Querying raw rows is insufficient, so we need a procedure that uses a PIVOT-style aggregation via cursor, collects the results into a PL/SQL associative array keyed by type, and prints a formatted summary table. This

combines associative arrays (INDEX BY VARCHAR2), nested loops, and multi-column group aggregation.

```
CREATE OR REPLACE PROCEDURE incident_statistics IS
TYPE CountMap IS TABLE OF NUMBER INDEX BY VARCHAR2(60);
  v_map      CountMap;
  v_key      VARCHAR2(60);
  v_type     INCIDENT.TYPE%TYPE;
  v_status   INCIDENT.STATUS%TYPE;
  v_count    NUMBER;
CURSOR stats_c IS
SELECT TYPE, STATUS, COUNT(*) AS CNT
FROM INCIDENT
GROUP BY TYPE, STATUS
ORDER BY TYPE, STATUS;
BEGIN
-- to load all grouped counts into the map
FOR rec IN stats_c LOOP
v_key := rec.TYPE || '|' || rec.STATUS;
v_map(v_key) := rec.CNT;
END LOOP;

-- Print header
DBMS_OUTPUT.PUT_LINE( RPAD('Type', 15) || RPAD('Status', 12) || 'Count');
DBMS_OUTPUT.PUT_LINE(RPAD('-', 35, '-'));
-- *RPAD = Right PAD

-- Iterate map in key order and print rows
v_key := v_map.FIRST;
WHILE v_key IS NOT NULL
LOOP
  v_type := SUBSTR(v_key, 1, INSTR(v_key, '|') - 1);
  v_status := SUBSTR(v_key, INSTR(v_key, '|') + 1);
  DBMS_OUTPUT.PUT_LINE( RPAD(v_type, 15) || RPAD(v_status, 12) ||
v_map(v_key));
  v_key := v_map.NEXT(v_key);
END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 35, '-'));
DBMS_OUTPUT.PUT_LINE('Report generated at: ' || TO_CHAR(SYSDATE, 'DD-MON-YYYY
HH24:MI:SS'));
END incident_statistics;
/

BEGIN incident_statistics;
END;
/
```

Type	Status	Count
Accident	Active	2
Burglary	Active	1
Fire	Active	1
Fire	Resolved	1
Medical	Active	2
Medical	Resolved	2
Report generated at: 07-MAY-2026 12:19:59		

Incident classification, team loading, and safe escalation package.

As the 112 system grows, the logic for classifying incidents, validating statuses, loading responder teams, and escalating stale cases must be reusable and easy to maintain. A PL/SQL package is the right tool. The package declares a VARRAY type (`t_status_list`) for the three allowed statuses and a Nested Table type (`t_responder_ids`) for dynamic responder lists. Function `get_priority` uses a CASE expression to map an incident type to HIGH, MEDIUM, or LOW, while `get_action` use one to translate priority into a dispatcher action. Procedure `load_team` fills a Nested Table with the responder IDs for a given incident and `escalate_pending` opens a FOR UPDATE cursor that locks every *Pending* row, calls `get_priority` and `get_action` for each one, validates the status against the VARRAY, and uses WHERE CURRENT OF to safely promote HIGH-priority incidents to *Active*. Finally, a master procedure (`run_report`) ties everything together and prints a formatted report.

```
CREATE OR REPLACE PACKAGE incident_mgmt AS
TYPE t_status_list IS VARRAY(5) OF VARCHAR2(20);
TYPE t_responder_id IS TABLE OF RESPONDER.ID%TYPE; -- nested table

FUNCTION get_priority(p_type IN VARCHAR2) RETURN VARCHAR2;
FUNCTION get_action (p_priority IN VARCHAR2) RETURN VARCHAR2;

PROCEDURE load_team (p_incident_id IN NUMBER, v_team OUT t_responder_id);
-- to lock pending rows and to classify, validate and escalate HIGH ones
PROCEDURE escalate_pending;
PROCEDURE run_report;

END incident_mgmt;
/

CREATE OR REPLACE PACKAGE BODY incident_mgmt AS
FUNCTION get_priority(p_type IN VARCHAR2) RETURN VARCHAR2 IS
BEGIN
    RETURN CASE p_type
        WHEN 'Medical' THEN 'HIGH'
        WHEN 'Fire'     THEN 'HIGH'
        WHEN 'Accident' THEN 'MEDIUM'
        WHEN 'Crime'    THEN 'MEDIUM'
        ELSE             'LOW'
    END;
END get_priority;

FUNCTION get_action(p_priority IN VARCHAR2) RETURN VARCHAR2 IS
BEGIN
    RETURN CASE p_priority
        WHEN 'HIGH' THEN 'ESCALATE NOW'
        WHEN 'MEDIUM' THEN 'QUEUE FOR REVIEW'
        ELSE          'LOG AND WATCH'
    END;
END get_action;

PROCEDURE load_team(p_incident_id IN NUMBER, v_team OUT t_responder_id) IS
BEGIN
    v_team := t_responder_id(); -- initialise empty nested table
    FOR rec IN (SELECT RESPONDER_ID FROM INCIDENT
                WHERE ID = p_incident_id)
```

```
    LOOP
    v_team.EXTEND;
    v_team(v_team.LAST) := rec.RESPONDER_ID;
    END LOOP;
END load_team;

PROCEDURE escalate_pending IS
v_allowed t_status_list := t_status_list('Pending', 'Active', 'Resolved');
CURSOR c_pend IS
    SELECT ID, TYPE, STATUS
    FROM    INCIDENT
    WHERE   STATUS = 'Pending'
    FOR UPDATE OF STATUS NOWAIT;

    v_priority    VARCHAR2(10);
    v_action      VARCHAR2(25);
    v_found       BOOLEAN;
    v_count       NUMBER := 0;

BEGIN
DBMS_OUTPUT.PUT_LINE(RPAD('ID',6)||RPAD('TYPE',12)||
RPAD('PRIORITY',10)||'ACTION');
DBMS_OUTPUT.PUT_LINE(RPAD('-',60,'-'));
FOR rec IN c_pend
    LOOP
    v_priority := get_priority(rec.TYPE);
    v_action   := get_action(v_priority);
    v_found    := FALSE;
    FOR i IN 1 .. v_allowed.COUNT
        LOOP
            IF rec.STATUS = v_allowed(i) THEN v_found := TRUE;
                EXIT;
            END IF;
        END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD(rec.ID,6)||RPAD(rec.TYPE,12)||RPAD(v_priority,10)||
v_action);

        IF v_priority = 'HIGH' AND v_found THEN
            UPDATE INCIDENT SET STATUS = 'Active'
            WHERE CURRENT OF c_pend;
            v_count := v_count + 1;
        END IF;
    END LOOP;

COMMIT;

DBMS_OUTPUT.PUT_LINE(RPAD('-',50,'-'));
DBMS_OUTPUT.PUT_LINE('Escalated: ' || v_count || ' incident(s).');

EXCEPTION WHEN OTHERS THEN ROLLBACK;
    DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END escalate_pending;
```

```
PROCEDURE run_report IS v_team t_responder_id;
BEGIN
DECLARE
v_list t_status_list := t_status_list('Pending','Active','Resolved');
    BEGIN
        DBMS_OUTPUT.PUT_LINE('=== Allowed statuses ===');
        FOR i IN 1 .. v_list.COUNT
            LOOP
                DBMS_OUTPUT.PUT_LINE('  [' || i || '] ' || v_list(i));
            END LOOP;
        END;

DBMS_OUTPUT.PUT_LINE(RPAD('-',60,'-'));

-- to load and print the responder team for incident 502
load_team(502, v_team);

DBMS_OUTPUT.PUT_LINE('Team for Incident 502 has ' || v_team.COUNT || '
responder(s) and the responders are: ');

FOR i IN 1 .. v_team.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(' - Responder ID: ' || v_team(i));
END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

escalate_pending;
END run_report;

END incident_mgmt;
/

BEGIN
    incident_mgmt.run_report;
END;
/
```

```
=== Allowed statuses ===
[1] Pending
[2] Active
[3] Resolved

-----
Team for Incident 502 has 1 responder(s) and the responders are:
- Responder ID: 304
-----
ID    TYPE      PRIORITY ACTION
-----
511   Fire        HIGH    ESCALATE NOW
-----
Escalated: 1 incident(s).
```

Coordinating Professor Drd. Dincă Laurențiu-Gabriel
CSIE - Faculty of Economic Informatics

Student: Smeu Estera-Valentina

Group: 1107 | 3rd year

21.05.2026